# Proyecto Integrador Introducción al desarrollo de software

Cátedra Lanzillotta

2do cuatrimestre 2026

El proyecto final de la materia **Introducción al Desarrollo de Software** tiene como objetivo integrar y aplicar los conocimientos adquiridos durante el cuatrimestre, así como promover la investigación y el aprendizaje autónomo de nuevos conceptos y tecnologías.

Este trabajo busca acercar a los estudiantes a un entorno de desarrollo profesional, reproduciendo las principales dinámicas y desafíos que se presentan en un proyecto de software real. Es por este motivo que la solución deberá ser desplegada, funcional y cumplir con los requisitos establecidos, **sin presentar errores críticos.**

Para este proyecto se deberán formar equipos de entre **8 y 10 integrantes**. Cada equipo deberá aplicar buenas prácticas de programación, utilizar metodologías ágiles para organizar y gestionar el trabajo, y **promover la participación activa y equitativa de todos sus integrantes.**

Desde el punto de vista técnico, la solución deberá contemplar el desarrollo de un **Backend, un Frontend y un Backoffice**, utilizar **MySQL** como sistema de gestión de bases de datos y mantener el código fuente versionado en un repositorio de **GitHub**. Asimismo, se deberán integrar y aplicar los conceptos, herramientas y tecnologías trabajados durante la cursada.

Para esta edición, la cátedra propone **dos temáticas posibles** para el desarrollo del proyecto. Como alternativa, los equipos podrán presentar una propuesta de temática o idea propia. Para los equipos que presenten una idea propia, la misma deberá ser aprobada, para esto enviar un mensaje privado de Slack al docente titular explicando la propuesta, y alcance **(fecha límite para entregar la idea es el lunes 19/10)**

El proyecto tiene como objetivo principal, el desarrollo colaborativo de una solución de software, desde su concepción hasta su implementación y despliegue.

# Propuestas

## 1- Sistema de venta de entradas de cine

### Contexto del negocio

**Cine Paradiso** es una empresa de entretenimiento que tiene al momento 3 centros de entretenimiento (locales) con salas de cines de capacidades distintas (butacas). Cine Paradiso quiere competir con las empresas de streaming y darle al usuario una experiencia diferente ofreciendo poder ver películas que fueron estrenadas años atrás.

El CTO de Cine Paradiso nos solicita la creación de una aplicación web que permita la compra de entradas de una manera ágil, rápida y segura.
Para esto nos envía un listado de requerimientos junto con una breve descripción del sistema.

### Descripción

Sistema web realizado en el framework Flask, debe contener un Front de acceso a usuarios finales, y un backoffice de acceso al admin que permita configurar cada una de las salas con la película correspondiente, además de acceder a estadísticas, por otra parte un backend con base de datos mysql. Todos los endpoints deben tener seguridad.

### Contexto

**¿Qué es una función?** Una **función** es una película proyectada en una sala determinada, en una fecha y horario puntual. Cada función tiene sus propias butacas disponibles.

**¿Qué es una butaca?** Una **butaca** es un asiento identificado dentro de una sala. Al comprar una entrada se asigna una butaca específica, que no puede venderse dos veces en la misma función.

**¿Qué es la cartelera?** La **cartelera** es el conjunto de películas y funciones vigentes que el cine ofrece al público.

### Requerimientos:

**Front:**

- Manejo de usuarios. Roles: administrador y cliente.
- Perfil de cliente con información sobre consumos y películas que ví anteriormente.
- Frontend con la cartelera: películas en cartel, sinopsis, póster, duración, clasificación y funciones disponibles.
- Acceso a la capacidad y disponibilidad de butacas en cada una de las salas
- Compra de entradas. Flujo ágil: película → función → selección de butacas (mostrando ocupadas y libres) → confirmación al email. Se genera un QR con los datos de la compra que se envía por mail, con botón cancelar.
- Compra de servicios. Esto puede incluir comida, bebidas, estacionamiento, acceso VIP, entre otras configuraciones.

- Checkout de la compra, con envio de factura al mail, junto a un QR que permitirá el ingreso a la sala
- Una butaca no puede venderse dos veces en la misma función. La cancelación solo se permite hasta un tiempo determinado antes de la función.

**Backoffice**

Posibilidad de cargar películas, imágenes y sinopsis indicando sala, capacidad, local y tiempo en que estará disponible la película.

Dashboards informativos dentro del rol administrador: ocupación por función y por película, ventas por género, franjas horarias más vendidas, cancelaciones.

Poder cargar, modificar y dar de baja servicios. Estos servicios están disponibles para todas las salas.

Validación en ingreso a la sala: el escaneo del QR verifica que corresponda a esa función y fecha, que no haya sido utilizado ni cancelado, y lo marca como utilizado.
El catálogo de películas disponibles se debe consumir de alguna API gratuita como ser https://www.omdbapi.com/ o https://developer.themoviedb.org/docs/getting-started

**Deseables (no obligatorios)**

- Puntuación o reseñas de películas, habilitadas solo para usuarios cuyo QR de esa función fue validado (es decir, que efectivamente asistieron).
- PDF con reporte de ventas por función y películas más vistas.
- Dockerizar el proyecto.
- Hostear el proyecto y la base de datos en alguno de los servicios gratuitos indicados en los requisitos generales deseables.

---

## 2- Sistema de turnos médicos con check-in por QR

### Contexto del negocio

**Centro Médico Salud Integral** es un instituto médico integral que cuenta con médicos de diversas especialidades (clínica médica, pediatría, cardiología, dermatología, traumatología, entre otras) y varios consultorios. Actualmente la gestión de turnos se realiza mediante agendas de papel, llamadas telefónicas y mensajes de WhatsApp, lo cual genera inconvenientes como:

- **Solapamientos** de turnos del mismo médico en el mismo horario.
- **Cancelaciones** que no se registran correctamente, dejando franjas horarias ociosas.
- Pacientes que llegan a la clínica sin un turno confirmado o con datos incorrectos.
- Una sala de espera desordenada: recepción no sabe qué pacientes llegaron ni en qué orden.
- **Ausentismo** alto y sin medición: pacientes que reservan y no se presentan, sin consecuencia ni registro.

- Falta de reportes que permitan mejorar la asignación de consultorios y horarios.

El CTO de este centro médico nos solicita una plataforma web que permita definir la agenda de cada médico, reservar turnos online, registrar la llegada de los pacientes mediante el escaneo de un QR y medir el ausentismo y la ocupación real.
Para esto nos envía un listado de requerimientos junto con una breve descripción del sistema.

## Descripción

Sistema web realizado en el framework Flask, debe contener un Front de acceso a pacientes, y un backoffice de acceso al admin que permita configurar las especialidades médicas, los profesionales médicos y los consultorios disponibles, además de acceder a estadísticas, por otra parte un backend con base de datos mysql. Todos los endpoints deben tener seguridad.

## Contexto

**¿Qué es un turno médico?** Un **turno** es una franja horaria asignada a un paciente para ser atendido por un médico en un consultorio.

**¿Qué es la agenda?** La **agenda** es la configuración semanal que indica en qué días y horarios atiende un médico. A partir de ella se generan los turnos disponibles.

**¿Qué es el check-in?** El **check-in** es el registro de llegada del paciente a la clínica, realizado escaneando el QR de su turno dentro de una ventana horaria permitida. Si el paciente no hace check-in en esa ventana, el turno se marca como ausente.

## Requerimientos:

**Front:**

- Manejo de usuario. Roles: administrador/recepcionista y paciente.
- Todos los endpoints deben tener seguridad.
- Reserva de turnos. Flujo ágil: especialidad → médico → día → horario → confirmación al email. Al reservar, se envía por mail un QR con los datos del turno y un botón cancelar.
- Check-in en la clínica. Al llegar, el paciente presenta su QR; recepción lo escanea y el turno pasa a estado "confirmado / en sala de espera", registrando la hora de llegada. Si el paciente no hace check-in dentro de la ventana permitida (por ejemplo, hasta 15 minutos después de la hora del turno), el turno queda como "ausente".
- Un médico no puede tener dos turnos superpuestos; un paciente no puede tener dos turnos activos simultáneos con el mismo médico.

**Backoffice**

- ABM de médicos, especialidades, consultorios, coberturas y pacientes.
- Estados del turno: `disponible`, `reservado`, `confirmado`, `atendido`, `cancelado`, `ausente`.
- Agenda de cada médico (días, horarios, duración de turno, consultorio asignado), a partir de la cual se generan los turnos disponibles. Las agendas pueden ser semanales o mensuales

- Dashboards informativos: sala de espera del día ordenada por llegada, turnos por médico y especialidad, estadísticas de ausentismo, ocupación de consultorios, tiempo de espera del paciente, etc.

**Deseables (no obligatorios)**

- Sobreturnos fuera de la agenda regular.
- Recordatorio automático por mail antes del turno.
- Rol médico con vista propia (marcar atendido, ver sus turnos del día).
- Dockerizar el proyecto.
- Hostear el proyecto y la base de datos en alguno de los servicios gratuitos indicados en los requisitos generales deseables.

---

### 3- Proyecto a elección

El mismo deberá tener un Frontend, Backoffice, un Backend, una base de datos MySQL o PostgreSQL. Debe ser validado por la cátedra; fecha límite de presentación de la propuesta: **Lunes 19/10**. La propuesta debe ser enviada por mensaje privado de Slack a Bruno.

#### Requisitos mínimos

Si el proyecto no cumpliera con alguno de estos requisitos, deberá ser validado por el docente:

- Login de usuarios, con distintos roles y vistas dentro de la aplicación.
- Dashboards informativos.
- Informes en PDFs.

#### Deseables (no obligatorios)

- Alguna funcionalidad que haga uso de códigos QR (generación y/o validación).
- Dockerizar el proyecto.
- Hostear el proyecto y la base de datos en alguno de los servicios gratuitos indicados en los requisitos generales deseables.

# Requisitos obligatorios para cualquiera de los proyectos

Para cualquiera de las ideas se deberán cumplir los siguientes requerimientos:

- Uso de GitHub. Deben compartirlo con el corrector de la cátedra y todos deben participar aportando commits y desarrollo útil. Plantear y mantener las ramas adecuadas a este tipo de proyectos.
- Utilización de un tablero Kanban con tareas a desarrollar y backlog de producto, utilizando GitHub Projects, Trello o Jira.

- Los hitos a presentar —según las fechas indicadas más abajo— deberán entregar valor al cliente.
- **TODOS** los integrantes del equipo deberán realizar commits al repositorio, según vaya avanzando el proyecto. Estos commits deberán estar asociados a una tarea y en el mensaje del commit se deberá informar claramente qué se está realizando o a qué tarea se refiere.
- Deberán utilizar ramas de acuerdo a lo aprendido en este curso.
- Utilizar una base de datos MySQL o PostgreSQL con al menos 3 tablas y un script de creación/carga.
- El manejo de fecha y hora debe realizarse en un único campo de tipo fecha/hora (`datetime`), y no en dos campos separados (uno de fecha y otro de horario).
- Tener una aplicación backend utilizando Flask corriendo que se conecte a la base de datos y cualquier otra dependencia. Los endpoints deberán ser realizados utilizando el "verbo" correcto y se utilizará el formato JSON para enviar o recibir información.
- Tener una aplicación para el frontend utilizando Flask que se conecte al backend respetando RESTful.
- Está terminantemente prohibido utilizar funcionalidades que generen peticiones al front o al back desde el navegador (`fetch`, `promises`, `axios`, `XMLHttpRequest` u otras equivalentes): la comunicación entre el frontend y el backend debe realizarse en Python, mediante la librería `requests`.
- Está terminantemente prohibido el uso de ORMs (SQLAlchemy, SQLModel, Django ORM, Peewee u otros), incluida la definición de clases en Python que mapeen tablas de la base de datos: la comunicación o consultas a la base de datos deben hacerse mediante sentencias SQL explícitas escritas en el código, que se ejecutan contra la base de datos.
- Generación de códigos QR desde la aplicación y mecanismo de validación/escaneo según lo especificado por cada temática (en el proyecto a elección, esta funcionalidad es deseable).
- La aplicación debe contemplar el acceso desde dispositivos móviles, mediante una de estas dos opciones: integrar una app mobile usando únicamente Kivy UI o, en caso de no optar por esta opción, que la página web sea responsive (se adapte correctamente a pantallas de celulares y tablets).
- Presentación de informe detallando las principales decisiones de diseño, dificultades encontradas y conclusiones.
- En caso de utilizar dependencias se deberá tener un archivo `init.sh` que tenga la instalación de las mismas.

## Requisitos generales deseables

- Documentación sobre cada endpoint de la API y ejemplos de uso (pueden utilizar swagger para esto).
- Hostear el proyecto en alguna opción gratuita (Vercel, Render, PythonAnywhere, Railway, Koyeb, entre otras) y utilizar una base de datos hosteada en algún servicio gratuito (Supabase, Neon, Aiven, TiDB Cloud, db4free, entre otros).

## Hitos del proyecto (entregables)

Las formas de realizar las entregas deberán acordarse con el tutor asignado. Las fechas de cada hito serán definidas por la cátedra:

**Hito 1 —26/10.**

a. Entrega de hipótesis o supuestos, alcance y backlog de producto, mockup del sitio web (pueden usar alguna app online o directamente un dibujo).
b. Esquema de tablas a utilizar.
c. Listado de endpoints a utilizar
d. Creación del repositorio

**Hito 2 — 02/11.**

a. Listado de servicios/endpoints desarrollados.
b. Tablas SQL a utilizar.
c. Templates desarrollados, en base a los mockups entregados.

**Hito 3 — 9/11.**

a. Integración de Front con el Backend.
b. Funcionalidad finalizada de login, acceso al Backoffice, etc

**Hito 4 — 18/11.**

Entrega funcional de la **primera versión** del proyecto.

**Hito 5 — 25/11.**

**Primera entrega del Proyecto Integrador y defensa**.

**Hito 6 — 30/11.**

**Defensas y consultas.**

**Hito 7 — 2/12.**

**Segunda entrega del TP integrador y defensa.**

## Defensa Oral

La entrega de este Proyecto estará acompañada de una defensa oral tanto grupal como individual. Esta se considera la instancia más importante de evaluación dentro de la materia.

Se espera que tanto el alumno como el grupo en su totalidad puedan responder de forma completa las preguntas que llegara a plantear el docente en lo que respecta a la funcionalidad del trabajo, las herramientas provistas por el lenguaje de programación y el diseño del sistema en general.

## Documentación Requerida

Para la presentación del proyecto, se pide al equipo la presentación de un informe que contenga:

- Carátula
- Integrantes
- Resumen
- Introducción
- Solución propuesta

- Tecnología utilizada
- Conclusión final

La información volcada en el documento será utilizada por el equipo docente previo a corregir el trabajo, por lo cual el alumno puede expresar allí lo que crea conveniente para ayudar en el proceso de corrección.

## Condiciones de Aprobación

NO se debe:

- usar variables globales
- usar ciclos infinitos
- escribir código repetido
- entregar un programa que no funcione
- entregar un programa con errores graves
- incumplir con la fecha de entrega

Sí se debe:

- Evitar que el programa rompa frente a la interacción con el usuario
- modularizar el código
- usar buenas prácticas de programación
- utilizar un repositorio en GitHub
- entregar un programa que cumpla con la funcionalidad completa especificada

## Fecha de Entrega

**El Proyecto cuenta con 2 fechas de entrega** (ver sección de hitos), con hora límite 18:00 hs. La primera fecha tiene posibilidad de recuperación; la segunda no.

Se espera que el grupo entregue el proyecto completo para la primera fecha, y —en caso de recibir correcciones por parte del equipo docente— tendrán posibilidad de realizar una segunda entrega en la segunda fecha. Esta primera instancia de entrega no restará puntos en caso de re-entregar en la segunda fecha. La segunda fecha definirá la nota del trabajo según corresponda.

Es importante saber que la primera entrega debe estar completa y sin errores graves. Quienes decidan entregar únicamente en la segunda fecha (sin pasar por la primera) no tendrán posibilidad de correcciones intermedias y tendrán una nota final del trabajo según corresponda. Tanto la primera entrega como la segunda deben contener la totalidad de la funcionalidad solicitada; no se aceptarán entregas parciales o trabajos incompletos en ninguna de las fechas.

## Importante

La aprobación de este proyecto es condición necesaria (junto con la aprobación del parcial) para regularizar la materia.